

Software Reuse

Soon Phei Tin - 2025

Topics

- Introduction to Software Reuse
- Benefits and Challenges of Software Reuse
- Key Software Reuse Strategies: -
 - Application Framework
 - Software Product Lines
 - Commercial Off-The-Shelf

Introduction to Software Reuse

- Why software reuse matter?
- Companies like Google and Netflix save millions by reusing code.
Today, we will learn how to do the same
- Definition: Software reuse is the practice of using existing software components (code, designs, etc.) to build new system

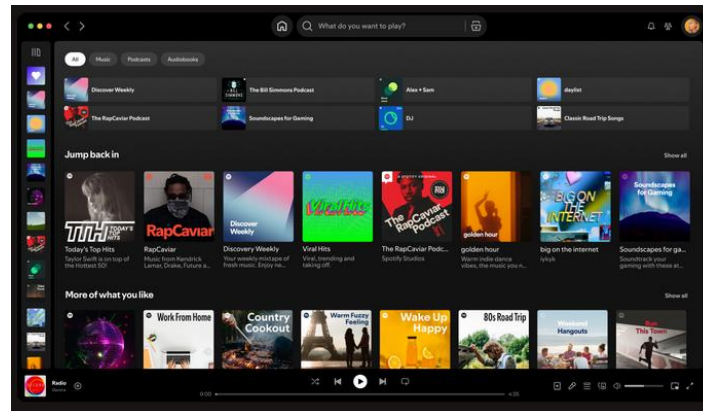
Introduction to Software Reuse

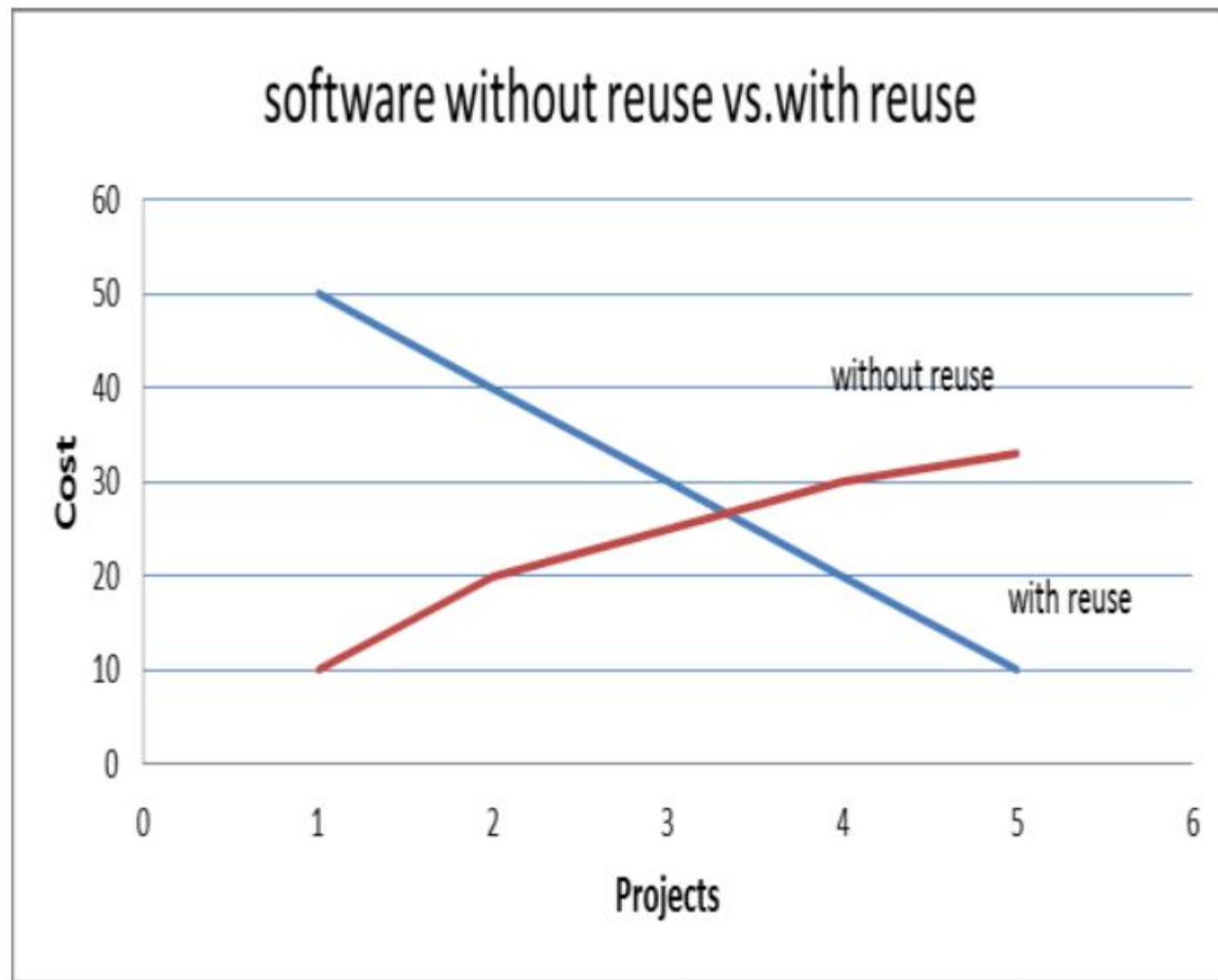
- Real-world impact: -
 - Faster development
 - TensorFlow powers multiple AI tools at Google
 - Google Launches TensorFlow Enterprise at No Extra Cost for Cloud Customers
 - Fewer bugs
 - React's reusable components reduce errors across Meta's platforms
 - Facebook, Instagram, and WhatsApp
- Why it's exciting: Reuse isn't just efficient – It's key skill in modern software engineering

Benefit of Software Reuse

- Why it saves time and money?
- Key benefit:
 - Cost savings: Reuse reduces development and maintenance cost
 - Faster delivery: Get products to market quicker
 - Higher quality: Tested components mean fewer bugs
 - Specialist knowledge: Leverage experts' work without reinventing the wheel

- Example: Spotify reuses UI components across its web and mobile apps for consistency and speed





Cumulative costs of software systems without reuse vs. with reuse Fig 2 illustrates that for a reuse oriented software development; there will be an initial cost increase. Then gradually cost will decrease due to reusing the same component again and again across similar products.

Velumani, P. Mangayarkarasi & Rangasamy, R.Selvarani. (2021). A Novel Software Cost Estimation Technique: Inclusion of Reusability.

Challenges of Software Reuse

- Why reuse isn't always easy?
- Key challenges:
 - Maintenance costs: Reuse code may become outdated or incompatible
 - Tool support: Some tools don't integrate well with reuse components
 - Not-invented-here syndrome: Developers may resist using others' code
 - Finding and Adapting components: It can be hard to locate and modify the right components
- Can you think of a time when reusing something didn't work out?

Challenges of Software Reuse

- Example:
 - **Ariane 5 Flight 501 (1996):** The Ariane 5 rocket exploded 40 seconds after launch due to a software error. The software reused a component from the Ariane 4 rocket without proper adaptation
 - <https://www.youtube.com/watch?v=N6PWATvLQCY>



(credit: ESA)



(credit: ESA)

Challenges of Software Reuse

- Example:
 - **Knight Capital Group (2012):** Knight Capital Group deployed new trading software that reused old code from a previous system. The old code was not properly deactivated, leading to a series of erroneous trades.
 - **Outcome:** The company lost \$440 million in just 45 minutes and nearly went bankrupt. This incident highlighted the risks of reusing legacy code without proper integration and testing.



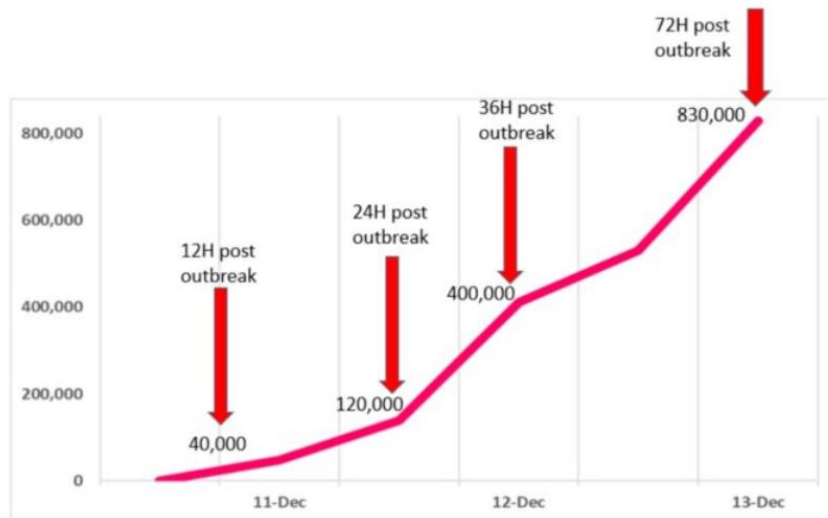
Jin Li | Bloomberg | Getty Images



by Bishr Tabbaa

Challenges of Software Reuse

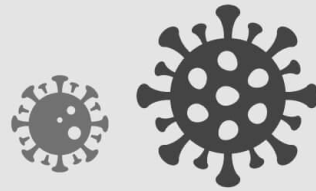
- Example:
 - **Log4j Vulnerability (2021):** The Log4j library, a widely reused open-source logging tool, had a critical vulnerability (CVE-2021-44228) that allowed remote code execution. Many organizations had reused Log4j in their systems without fully understanding its dependencies



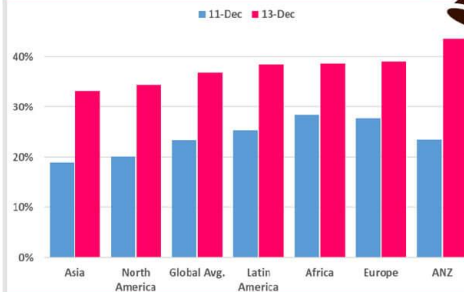
(credit: Check Point)

Log4j - A TRUE CYBER PANDEMIC

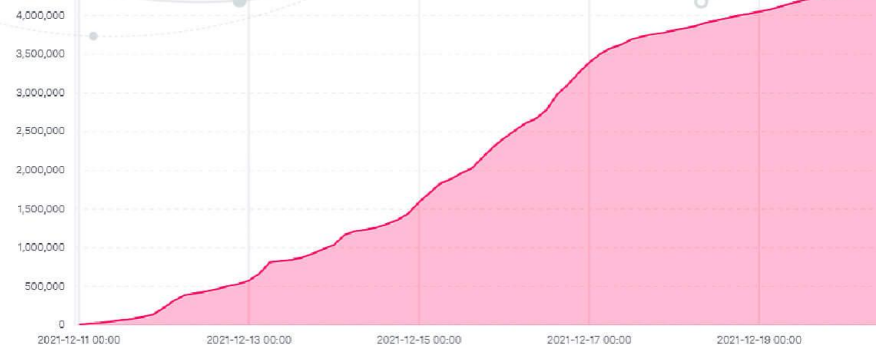
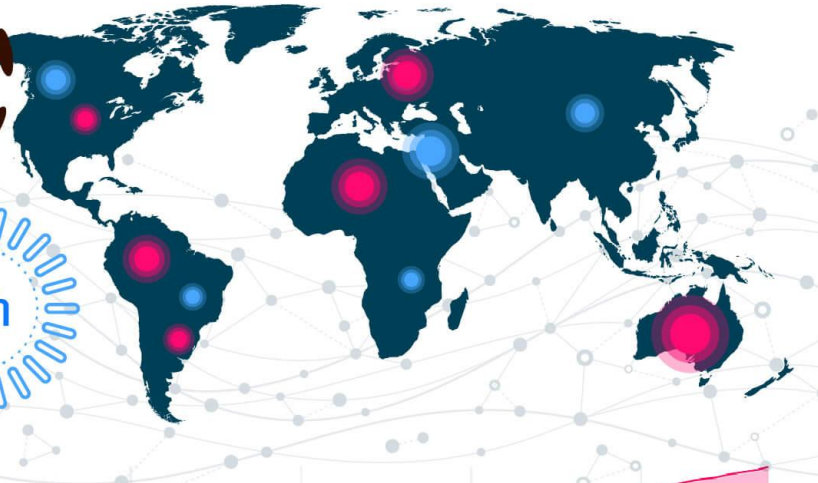
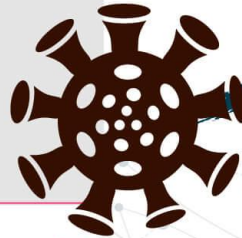
It is clearly one of the most serious vulnerabilities on the internet in recent years. When we discussed the Cyber pandemic, this is exactly what we meant – quickly spreading devastating attacks.



% Corporate Networks impacted per region



New variations of the original exploit being introduced rapidly - over 60 in less than 24 hours



Check Point prevented over 820,000 attack attempts since the outbreak

We have so far seen an attempted exploit on over 40% of corporate networks globally

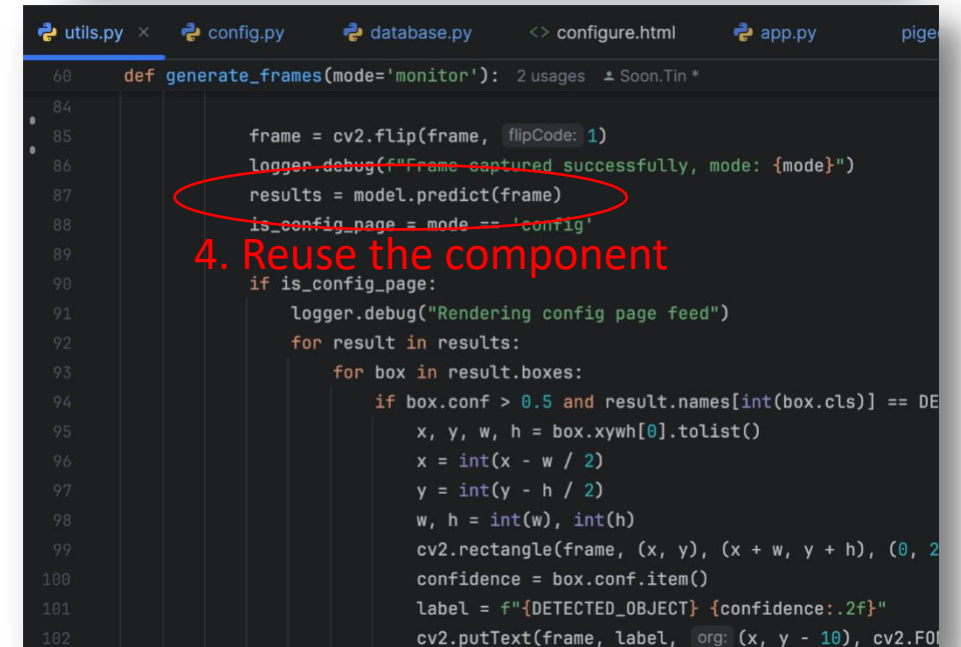
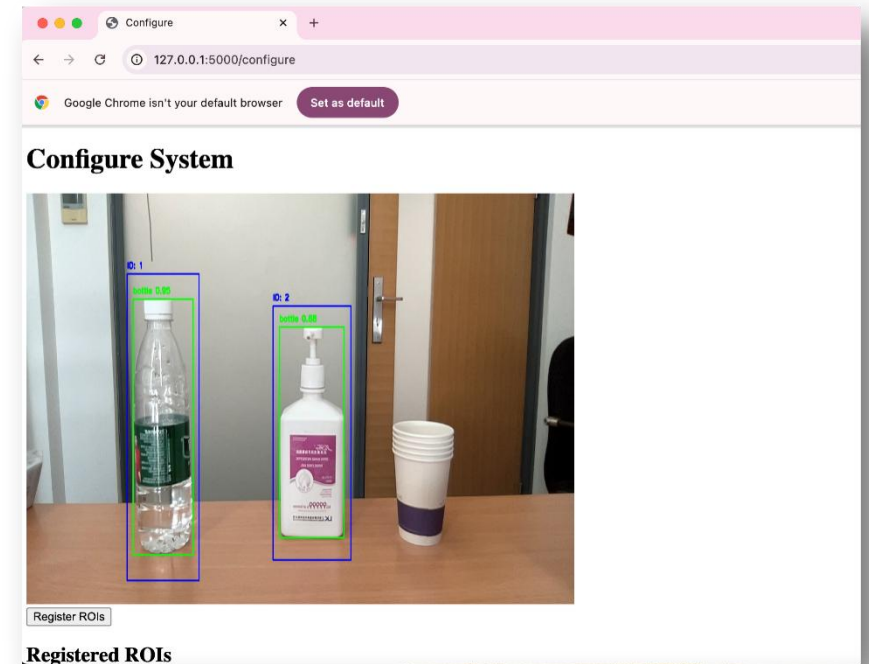
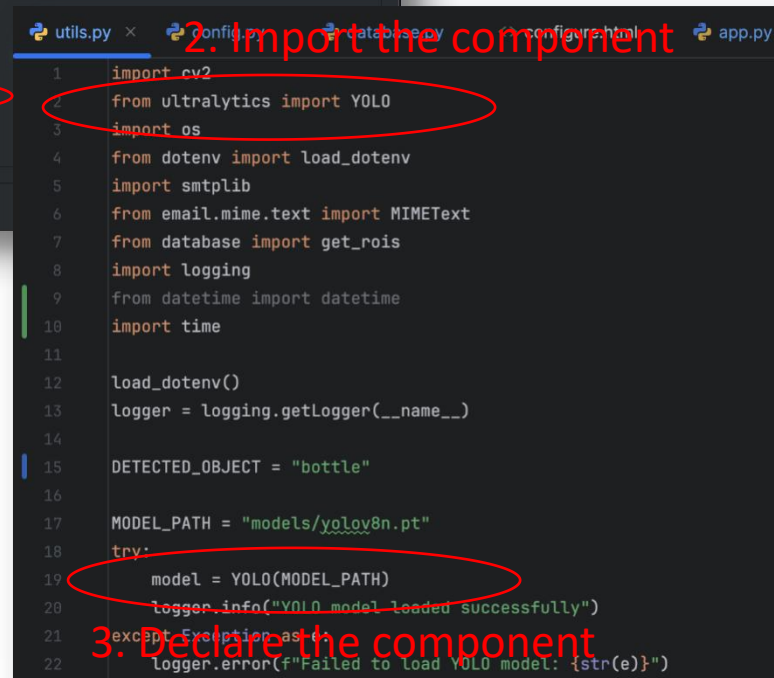
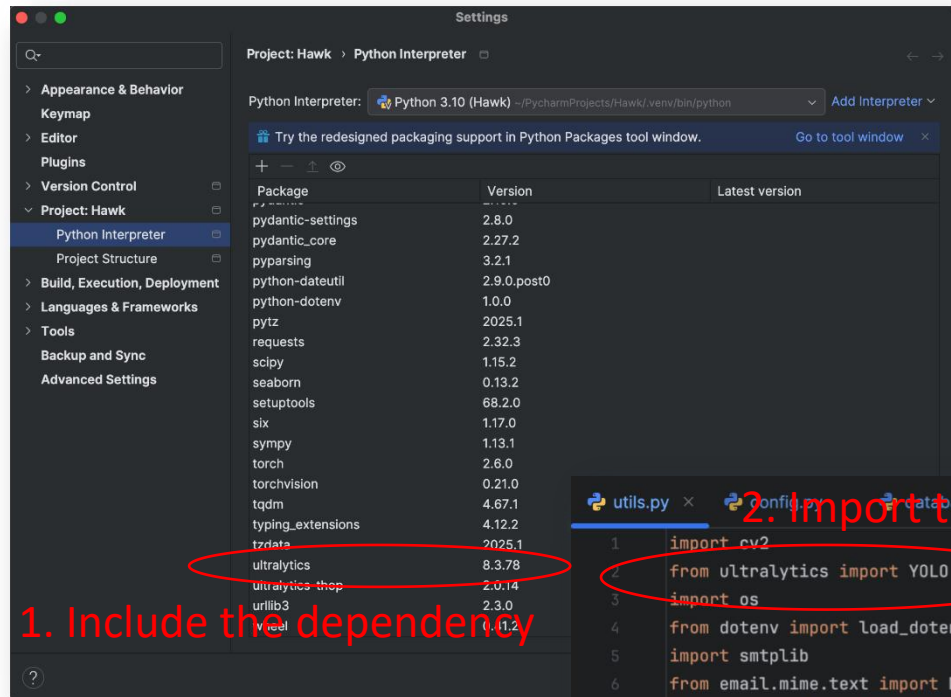


The data used in this report was detected by Check Point Software's Threat Prevention technologies, stored and analyzed in Check Point ThreatCloud. ThreatCloud provides real-time threat intelligence derived from hundreds of millions of sensors worldwide, over networks, endpoints and mobiles. The intelligence is enriched with AI-based engines and exclusive research data from Check Point Research – The intelligence & research arm of Check Point Software Technologies.

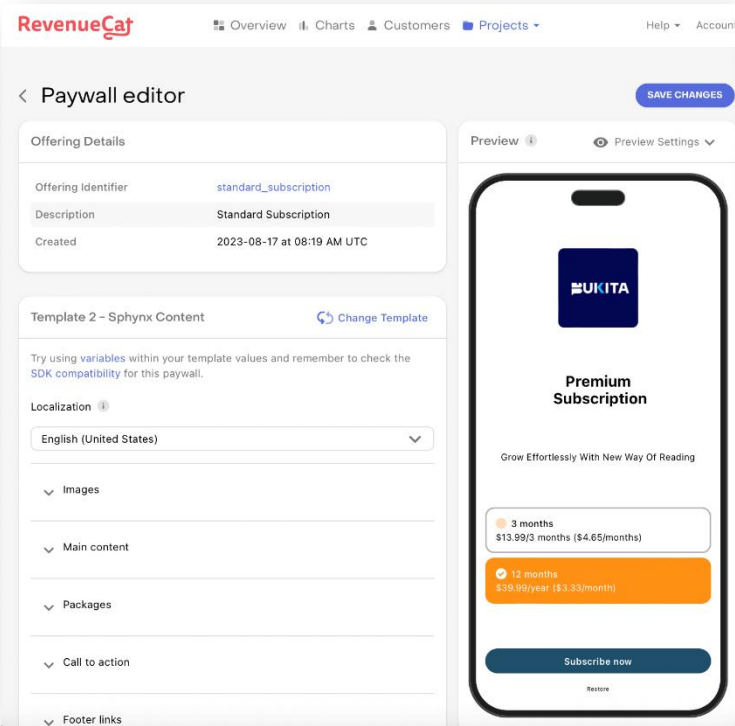
Core Reuse Techniques

- Key approaches:
 - Application framework: Pre-built structures (e.g. Django, Spring) that you extend.
 - Software produce lines (SPL): Families of similar products with shared components (e.g. Salesforce CRM variants)
 - COTS (Commercial Off-The-Shelf): Ready-made software adapted for specific needs (e.g. WordPress for websites)
 - Other: Component-based, open-source libraries, design patterns, and more

Hands-On: YOLO in Object Classification



Hands-On: Reuse in Action



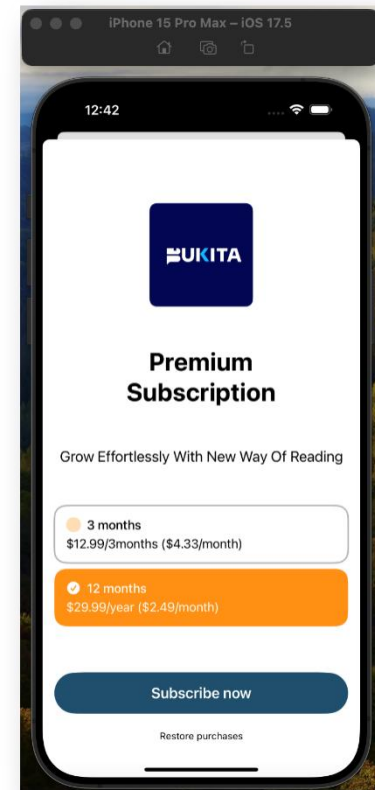
1. Define the Paywall

2. Include the dependency

```
! pubspec.yaml
24 dependencies:
52 path_provider: ^2.0.14
53 webview_flutter: ^4.2.0
54 new_version_plus: ^0.0.10
55 in_app_update: ^4.1.3
56 mask_text_input_formatter: ^2.4.0
57 hive: ^2.2.3
58 hive_flutter: ^1.1.0
59 format: ^1.3.0
60 moment_dart: ^0.17.4
61 timeago: ^3.4.0
62 google_sign_in: ^6.2.1
63 sign_in_with_apple: ^5.0.0
64 device_info_plus: ^9.0.3
65 flutter_spinkit: ^5.2.0
66 firebase_auth: ^4.20.0
67 firebase_core: ^2.32.0
68 package_info_plus: ^4.2.0
69 flutter_facebook_auth: ^6.0.4
70 the_apple_sign_in: ^1.1.1
71 firebase_messaging: ^14.9.4
72 flutter_local_notifications: ^16.2.0
73 flutter_app_badge_control: ^0.0.1
74 colorful_safe_area: ^1.0.0
75 iconsax: ^0.0.8
76 intl_phone_number_input: ^0.7.4
77 flutter_rating_stars: ^1.1.0
78 flutter_widget_from_html: ^0.9.1
79 lottie: ^1.4.3
80 media_info: ^0.12.0+2
81 intl: ^0.19.0
82 purchases_flutter: ^6.29.4
83 audio_video_progress_bar: ^2.0.3
84 purchases_ui_flutter: ^6.29.4
85 flutter_easyloading: ^3.0.5
86 sizer: ^2.0.15
87 flutter_native_splash: ^2.2.16
```

3. Reuse the component

```
lib > screens > profile > components > subscription_block.dart
18 class _SubscriptionBlockState extends State<SubscriptionBlock> {
57 Widget _memberBanner(){
58   return SizedBox.shrink();
59 }
60
61 Widget _subscriptionBanner() {
62   return GestureDetector(
63     behavior: HitTestBehavior.opaque,
64     onTap: () async {
65       await PurchasesService.makePurchase();
66     },
67     child: Container(
68       width: double.infinity,
69       // height: 94,
70       padding: const EdgeInsets.all(16),
71       clipBehavior: Clip.antiAlias,
72       decoration: ShapeDecoration(
```



4. End result

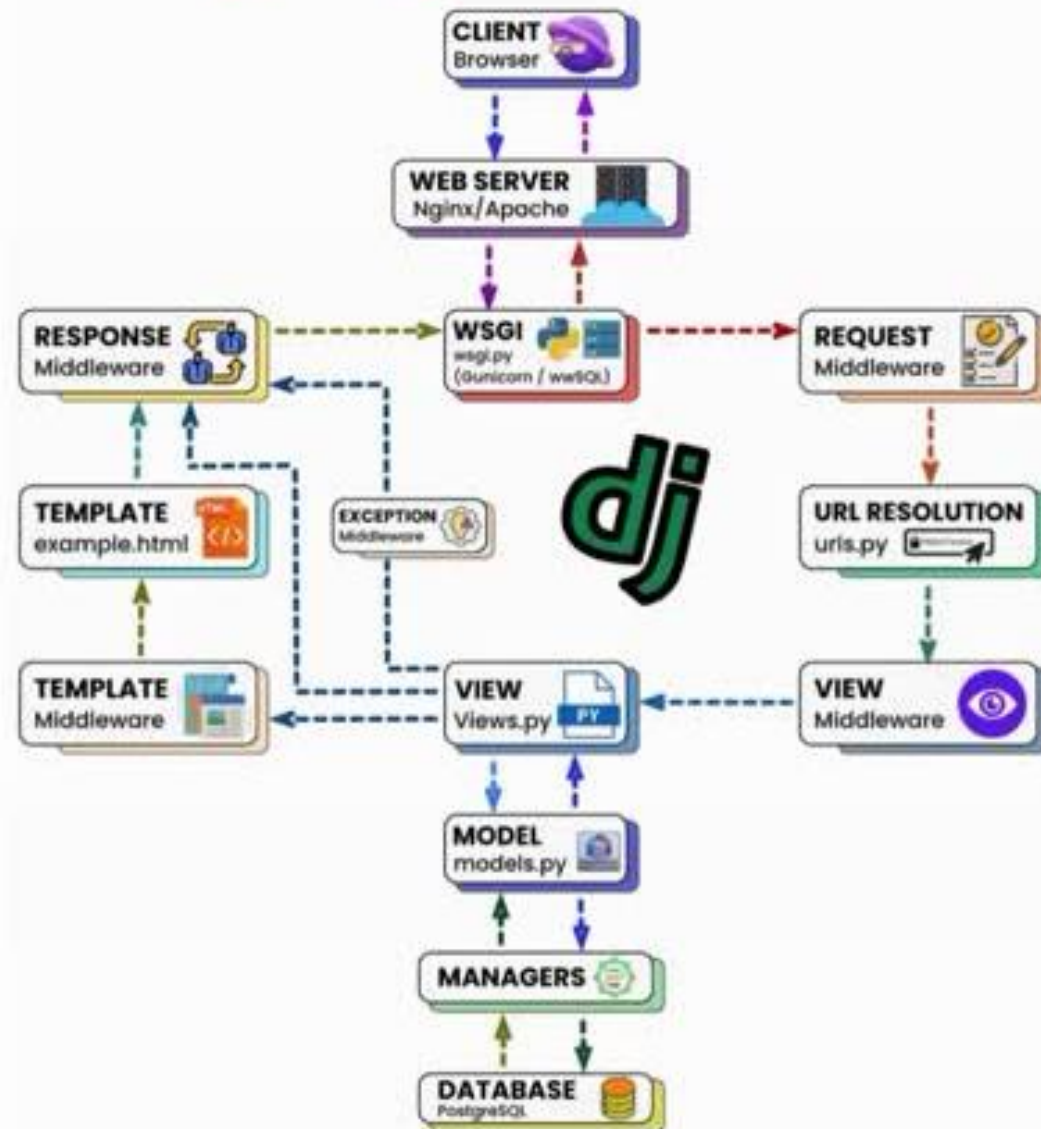
Application Framework

- Application framework: Build on a solid foundation
- Definition: A generic structure (e.g. a set of classes) that you extend to create specific applications
- Examples:
 - Web framework: Django (Python), Spring (Java)
 - Mobile framework: Flutter (cross-platform apps)

Application Framework

- Frameworks are like pre-built foundations. You save time from starting from the scratch by adding your customize features to the foundations.
- How they work: Provide common features (e.g., security, database support) that you customize.
- Analogy: Think of a framework as a house's basic infrastructure such as plumbing, electricity, roofing and wall. You can design the details and add your own fitting.

Django Request - Response Cycle



Extending Framework

- Extension methods:
 - Inheritance: Create subclasses from framework classes.
 - Callbacks: Register functions to be called at specific times or events.
 - Hooks: Insert custom code at predefined points.

Mini Research

- Imagine you're building a dynamic web app—say, a live music playlist editor—using a framework like React or Django. Your app needs to respond to user actions and lifecycle events. Dive into the roles of **callbacks** and **hooks**: how do they work in a web framework, and what sets them apart in bringing your app to life?

Extending Framework

- Example for the implementation of hook in Spring framework

```
import org.springframework.security.core.Authentication;
import org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import org.springframework.stereotype.Component;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@Component
public class CustomAuthenticationSuccessHandler implements AuthenticationSuccessHandler {

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response,
                                     Authentication authentication) throws IOException, ServletException {
        // Custom logic after successful authentication
        System.out.println("User " + authentication.getName() + " logged in successfully!");

        // Redirect to a specific page after login
        response.sendRedirect("/home");
    }
}
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private CustomAuthenticationSuccessHandler customAuthenticationSuccessHandler;

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
                .antMatchers("/public/**").permitAll() // Public endpoints
                .antMatchers("/admin/**").hasRole("ADMIN") // Restricted to ADMIN role
                .anyRequest().authenticated() // All other requests require authentication
                .and()
            .formLogin()
                .loginPage("/login") // Custom login page
                .successHandler(customAuthenticationSuccessHandler) // Hook for successful login
                .permitAll()
                .and()
            .logout();
    }
}
```

Challenges of Application Framework

- **Learning Curve:** Steep initial effort to understand and adapt the framework (e.g., mastering Django's structure).
- **Overhead:** May include unused features, bloating the app (e.g., extra libraries in React).

Application Framework vs. Libraries

- **Application Frameworks:**
 - Structure for entire apps (e.g., Django).
 - Controls flow—you extend it.
- **Software Libraries:**
 - Tools for specific tasks (e.g., Requests).
 - You control flow—call as needed.

	Application Framework	Software Library
Definition	A pre-built, reusable structure or skeleton that provides a foundation for developing applications. Django (web apps), Spring (enterprise apps)	A collection of reusable functions, classes, or utilities that developers call to perform specific tasks NumPy (math operations), jQuery (DOM manipulation)
Control Flow	Uses an inversion of control (IoC) model—<i>the framework controls the flow</i>. Developers plug their code into predefined entry points (e.g., hooks, callbacks) dictated by the framework.	The developer retains control—you <i>call the library</i> when you need its functionality
Scope and Scale	Broad scope, often covering an entire application domain	Narrower scope, focusing on specific tasks or features
Customization	Customization happens within the framework's constraints—developers extend or override its predefined behaviors	More flexible—developers use only what they need and integrate it however they want, with no imposed architecture.
Dependency	Your application depends heavily on the framework; it's the backbone of the system, and removing it would require a major rewrite.	Dependency is lighter—you can swap out or remove a library with less impact, as it's not integral to the app's structure.

Software Product Lines (SPL)

- Building a family of products
- Definition: A set of related software products sharing a common core but specialized for different needs.
- Example: Salesforce offer CRM variant for sales, support, and marketing all built from one SPL

Software Product Lines (SPL)

- Benefits:
 - Reuse across multiple products
 - Faster development for new variants
- Analogy: Car models with different features, but using the same engine under the hood
- SPLs are ideal when you need to create similar products with slight variations. The reuse in SPL is at the domain level.

SPL Specializations

Type of specialization	Description	Example
Platform specialization	Adapt for different devices	Web, mobile, embedded
Functional specialization	Add features for specific users	Sales, support
Environment specialization	Adjust for different operating environment	High-security, standalone
Process specialization	Support different workflow	Cash payment, credit card, debit card

Challenges for SPL

- **Upfront Investment:** High cost and time to design a reusable core (e.g., defining shared CRM features).
- **Complexity:** Managing variants can get messy as the family grows (e.g., updating all apps consistently).

Exercise

- A software development team is tasked with creating a fitness tracking application that supports multiple user roles (athletes, coaches, health professionals), includes workout planning features, and integrates with external wearable devices (e.g., Fitbit). The client demands delivery within five months and prioritizes maintainability.
- Complete the following structured short-answer questions based on the scenario:
 1. Name **one** software reuse technique suitable for this project.
 2. State **two reasons** why the technique you chose in part 1) is suitable for this project.

Commercial Off-The-Shelf (COTS)

- Ready-made software for quick deployment
- Definition: Pre-built software that you can buy and adapt without changing the source code
- Example: WordPress for websites, HubSpot for CRM, SAP for ERP

Commercial Off-The-Shelf (COTS)

- COTS can be tailored by
 - Configure settings
 - Plugins
 - API calls
- Benefits:
 - Fast deployment
 - Lower development risk
 - Vendor support

COTS vs. SPL

COTS	SPL
• Single product for broad use • Limited customization • Ready to use	• Family of products for specific domains • Highly customizable • Requires upfront works

Customer don't buy the SPL; They buy specific products from the family

Salesforce Case Study

- Internally, Salesforce develop its range of products using SPL strategy
- Market its products as individual COTS product to external customers
- An SPL could be packaged and sold as COTS products
- However, SPL is not always sold as COTS products

Testing COTS Systems

- Testing of COTS focus on integration
- Key focus: To test how the COTS software interacts with your operational processes
- COTS is pre-tested, but integration with your operation may not be smooth
- Example:
 - Ensure the COTS CRM integrates well with your existing email system
 - Ensure the COTS CRM works coherently with your process to manage customers

Challenges for COTS

- **Limited Customization:** Configuration-only approach restricts flexibility (e.g., WordPress can't be fully rewritten).
- **Vendor Dependency:** Reliant on vendor updates or support (e.g., delays in fixing SAP bugs).

Exercise

You are a software engineering consultant hired by a company that develops mobile applications for different industries, such as healthcare, finance, and education. The company is considering adopting software reuse techniques to reduce development time and costs while maintaining high quality. They are particularly interested in understanding the differences between **Application Frameworks** and **Software Product Lines (SPL)**.

Scenario:

The company plans to develop a new set of mobile apps for small businesses, including:

- A point-of-sale (POS) app for retail stores.
- An appointment scheduling app for service-based businesses.
- An inventory management app for warehouses.

All apps need to run on both iOS and Android, and they must share common features like user authentication, payment processing, and data analytics.

Exercise

- Tasks:
 - Define **Application Frameworks** and **Software Product Lines (SPL)**, highlighting their key differences.
 - Recommend which technique (Application Frameworks or SPL) is more suitable for the company's needs. Justify your choice with at least three specific reasons based on the scenario.
 - Identify two potential challenges the company might face when implementing your recommended technique.

Discussion

- For each strategy (Application Framework, SPL, and COTS), explain how it tackles *at least two* of the essential difficulties discussed in the essay written by Fred Brooks “No Silver Bullet”, using examples to show your strategy in action.
- Your task is to challenge the below given answer.

Discussion Answer

- **Application Frameworks:**

- **Complexity:** Frameworks like Django reduce complexity by providing a structured blueprint—pre-built modules for carts, payments, and user accounts simplify the app's design. Instead of juggling chaotic code, you follow a clear MVC pattern.
- **Changeability:** They handle evolving needs well; adding a new payment gateway (e.g., Stripe) is just a matter of plugging into Django's extensible architecture via middleware or apps.

Discussion Answer

- **Software Product Lines (SPL):**

- **Complexity:** SPLs tame complexity by creating a reusable core (e.g., shared checkout and inventory logic) for variants like mobile and desktop e-commerce apps, reducing redundant design work.
- **Conformity:** They align with real-world constraints by specializing the core—e.g., a mobile version conforms to touch inputs, while desktop supports keyboard navigation, all from one base.

Discussion Answer

- **COTS:**

- **Complexity:** COTS like Shopify slashes complexity—you get a pre-built e-commerce solution with carts and analytics out of the box, no need to design from scratch.
- **Changeability:** It adapts to changes via configuration (e.g., adding a discount feature through plugins), though major shifts might hit customization limits.

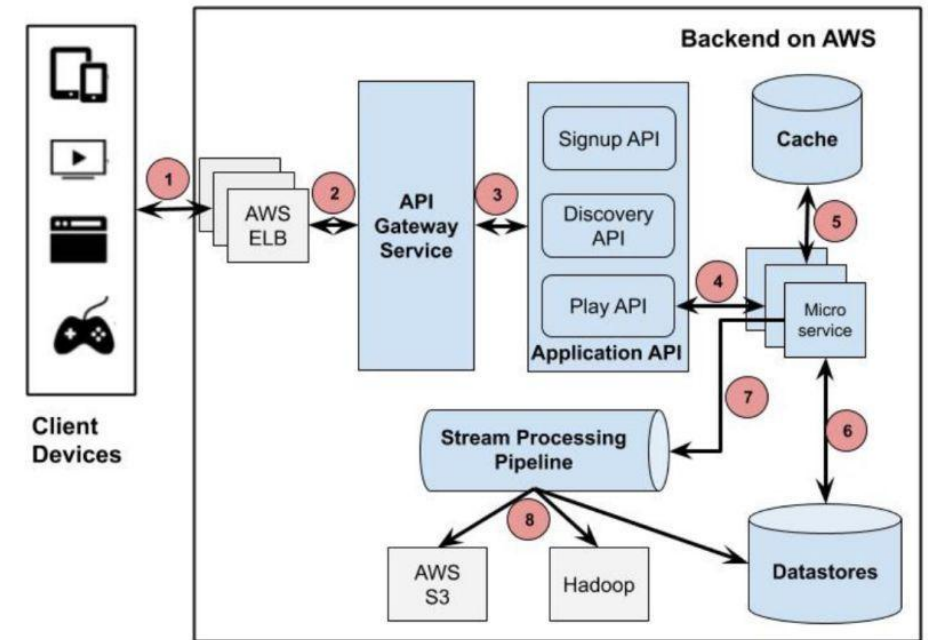
Reuse in the Real World

- Overview: Netflix reuses micro-services across its platform
- Benefit:
 - Scalability: Reuse services enable Netflix to
 - Handle million of users by adding more micro-service instances to a specific Netflix service
 - Easily add new Netflix service using existing micro-services
 - Reliability: Tested micro-services can be reused without testing again

Reuse in the Real World

- Scalability means growing without breaking
- Reuse micro-services enable Netflix to add more service instances and servers without disrupting users
- Flexibility: Deploy and scale individual services (e.g., adding more streaming instances during peak times) without affecting others.

Microservices Architecture at **NETFLIX**



Credits: Cao Duc Nguyen

Reuse in the Real World

- Around 2016-2017, Netflix was reported to have approximately **700 microservices**, as noted in articles from the Netflix TechBlog.
- A 2023 article from the Netflix TechBlog and a GeeksforGeeks post from 2024 suggest that Netflix now operates with **hundreds, if not thousands**, of microservices
- Based on the best available data and logical extrapolation, Netflix likely has **approximately 1,000 to 1,500 microservices**

Why Reuse Matters

- Summary
 - Reuse saves time, money, and effort.
 - Techniques like frameworks, SPL, and COTS offer different ways to reuse.
 - Real-world companies rely on reuse to stay competitive.
- As software engineers, mastering reuse will make you more efficient and valuable.
- How can you apply reuse in their own projects?

The End